Lab 2: Hidden Markov Models

Your Name

2026-09-22

Question 1

Build a hidden Markov model (HMM) for the scenario described

The model has ten hidden states and ten possible observations. The states form a ring and from each state, the process either remains in current state or moves to next state, both with a probability of 0.5. Each state reports its own sector and the two neighboring sectors on either side with equal probability. The initial state is uniformly distributed.

The HMM was initialized taking these constraints into account.

Question 2

Simulate the HMM for 100 time steps.

100 hidden states and observations was simulated from the HMM. The observations are noisy because each hidden state can emit five different symbols.

```
## $states
##   [1]  9  9  9  9 10  1  2  2  2  2  3  3  4  4  4  4  4  4  4  5  6  6  7  8  9
##  [26] 10 10 10  1  2  2  3  3  4  4  4  5  5  5  6  7  7  8  9 10  1  2  3  3  4
##  [51]  5  5  6  6  7  7  8  8  8  8  9 10 10 10 10  1  1  2  2  2  2  2  3  3  3
##  [76]  4  5  5  5  6  7  8  8  8  8  8  9  9  9 10 10 10  1  1  1  1  1  1  2  3
##
## $observation
##   [1]  7 10  8 10  2  3 10  3  4  4  5  4  2  3  2  6  6  5  4  3  5  5  8  9 10
##  [26]  9  9 10  2 10  2  5  3  2  6  6  4  7  7  6  5  9  7 10 10  3  3  1  3  3
##  [51]  6  5  4  7  7  9  9 10  6  9 10  2  9  9  8  1  3  2  3  4  3  2  5  4  4
##  [76]  2  4  6  4  6  8 10  8  7  6  6  7  8  9 10  1  9  2  2  3  9  2 10  4  1
```

Question 3

Compute the filtered and smoothed probability distributions for each of the 100 time points. Compute also the most probable path.

The filtered distribution was computed using the forward algorithm. The smoothed distribution was calculated by combining the forward and backward probabilities. The most probable path was found using the Viterbi algorithm.

Question 4

Compute the accuracy of the filtered and smoothed probability distributions, and of the most probable path.

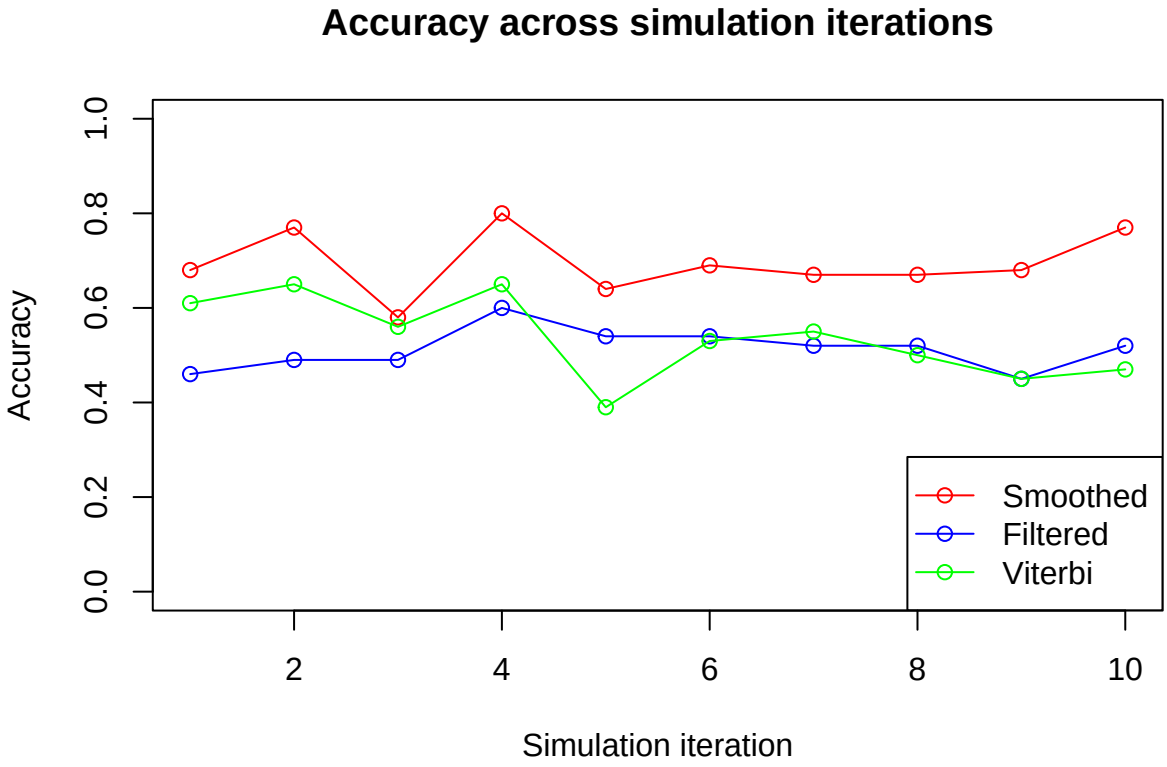
Each method was evaluated by comparing its most probable state at every time point with the true hidden state from the simulation.

```
##     method accuracy
## 1 Filtered     0.53
## 2 Smoothed     0.74
## 3  Viterbi     0.56
```

Question 5

Repeat the previous exercise with different simulated samples.

The three methods were evaluated over ten simulated iterations. The plot below shows how their accuracies vary between simulations.



In general, the smoothed distributions should be more accurate than the filtered distributions. Why? In general, the smoothed distributions should be more accurate than the most probable paths, too. Why?

Smoothing uses the complete observation sequence, $P(X_t \mid Y_{1:100})$, including future observations. It has more information than filtering, $P(X_t \mid Y_{1:t})$, and generally estimates each state more accurately.

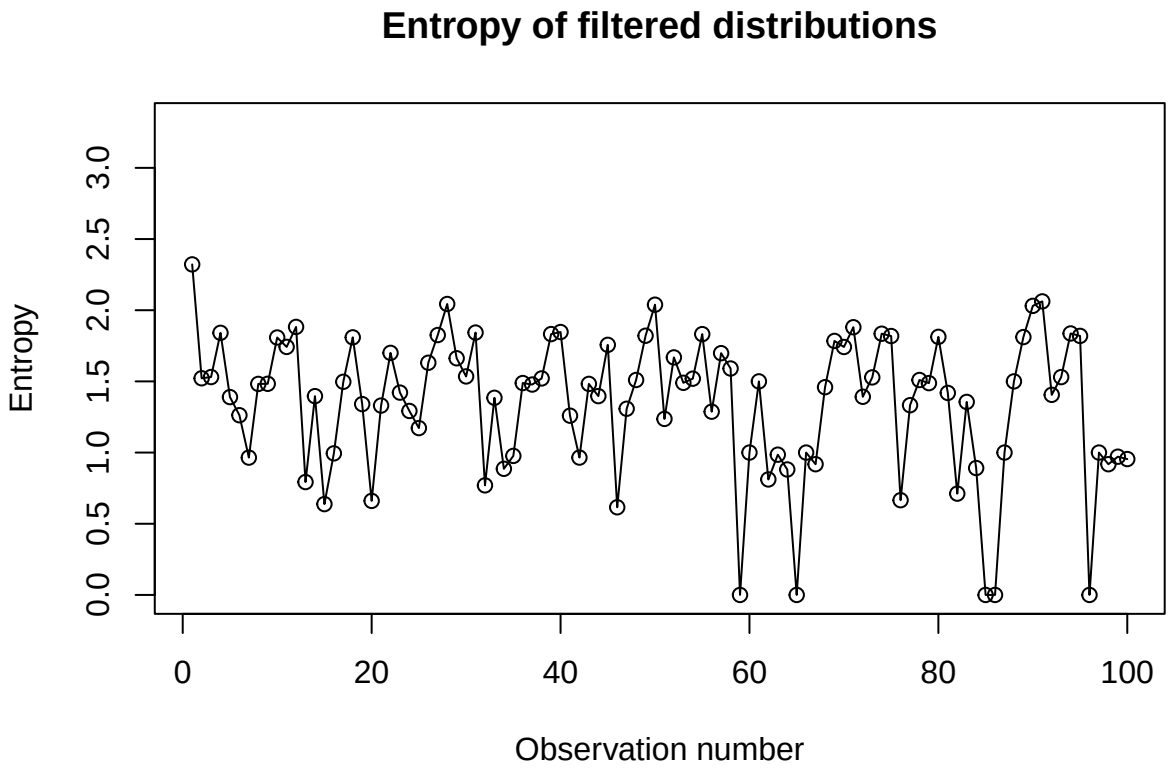
Smoothing selects the most probable state independently at each time point. Viterbi however, finds the single most probable path while respecting transition constraints. Therefore, the Viterbi path is not optimal for pointwise accuracy, which generally make its accuracy lower in this scenario.

Part 6: Entropy of the filtered distributions

Is it always true that the later in time the better you know where the robot is?

The entropy of each filtered state distribution measures uncertainty about the hidden state after the observations available at that time. The result is plotted below, and we can see that it is not true that the

later in time the better we know where the robot is. The entropy does not consistently decrease since a new observation can be ambiguous (the robot reports 5 different sectors) and increase uncertainty.



Part 7

Compute the probabilities of the hidden states for the time step 101.

The filtered distribution at time 100 was propagated through the transition matrix to calculte the distribution of X_{101}.

```
# The filtered distribution at time 100
filtered_prob_t100 <- filtered_probs[, 100]

prob_t101 <- drop(filtered_prob_t100 %*% trans_probs)
print(prob_t101)
```

```
##  [1] 0.0000 0.1875 0.5000 0.3125 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
```

Appendix A: R Code

Question.1

```
# Define ten hidden states and ten observable symbols
states <- 1:10
symbols <- 1:10

# Build the transition matrix
trans_probs <- matrix(0, nrow = 10, ncol = 10)
for (i in 1:10) {
  # Set probability to stay in current sector to 0.5
```

```
  trans_probs[i, i] <- 0.5
  # Set probability to move to next sector to 0.5
  trans_probs[i, if (i == 10) 1 else i + 1] <- 0.5
}

# Build the emission matrix
emis_probs <- matrix(0, nrow = 10, ncol = 10)
# Observe any of the 5 sectors [i-2, i+2] with equal probability
# Since it's a ring, the window wraps around
for (i in 1:10) {
  for (d in -2:2) {
    j <- ((i + d - 1) %% 10) + 1  # wrap-around for the ring
    emis_probs[i, j] <- 0.2
  }
}

# Uniform distribution for the initial hidden state
start_probs <- rep(0.1, 10)

# Create the hidden Markov model
hmm <- initHMM(
  States = states,
  Symbols = symbols,
  startProbs = start_probs,
  transProbs = trans_probs,
  emissionProbs = emis_probs
)
```

Question.2

```
# Simulate 100 hidden states and observations from the HMM
set.seed(12345)
sim <- simHMM(hmm, 100)
print(sim)

observations <- sim$observation
```

Question.3

```
# Compute the forward and backward log-probabilities
log_forward_probs <- forward(hmm, observations)
log_backward_probs <- backward(hmm, observations)

# Filtering: P(x_t | y_1:t)
filtered_probs <- exp(log_forward_probs)
# Normalize each time points distribution to sum to 1
filtered_probs <- sweep(filtered_probs, 2, colSums(filtered_probs), "/")

# Smoothing: P(x_t | y_1:100)
smoothed_probs <- exp(log_forward_probs + log_backward_probs)
# Normalize each time points distribution to sum to 1
smoothed_probs <- sweep(smoothed_probs, 2, colSums(smoothed_probs), "/")
```

```r
# Most probable path using the viterbi algorithm
viterbi_path <- viterbi(hmm, observations)
```

Question.4

```r
accuracy <- function(y, y_hat) {
  mean(y == y_hat)
}

# Get the most probable state at each time step
filtered_preds <- apply(filtered_probs, 2, which.max)
smoothed_preds <- apply(smoothed_probs, 2, which.max)

# Actual states
true_states <- sim$states

# Calculate accuracy
filtered_accuracy <- accuracy(true_states, filtered_preds)
smoothed_accuracy <- accuracy(true_states, smoothed_preds)
viterbi_accuracy <- accuracy(true_states, viterbi_path)

print(data.frame(
  method = c("Filtered", "Smoothed", "Viterbi"),
  accuracy = c(filtered_accuracy, smoothed_accuracy, viterbi_accuracy)
))
```

Question.5

```r
# Turning exercise 2 and 3 into a single function
calculate_accuracy <- function(hmm) {
  sim <- simHMM(hmm, 100)
  observations <- sim$observation

  log_forward_probs <- forward(hmm, observations)
  log_backward_probs <- backward(hmm, observations)

  # Filtering: P(x_t | y_1:t)
  filtered_probs <- exp(log_forward_probs)
  # Normalize each time points distribution to sum to 1
  filtered_probs <- sweep(filtered_probs, 2, colSums(filtered_probs), "/")

  # Smoothing: P(x_t | y_1:100)
  smoothed_probs <- exp(log_forward_probs + log_backward_probs)
  # Normalize each time points distribution to sum to 1
  smoothed_probs <- sweep(smoothed_probs, 2, colSums(smoothed_probs), "/")

  # Most probable path using the Viterbi algorithm
  viterbi_path <- viterbi(hmm, observations)

  # Get the most probable state at each time step
  filtered_preds <- apply(filtered_probs, 2, which.max)
  smoothed_preds <- apply(smoothed_probs, 2, which.max)
```

```r
  # Actual states
  true_states <- sim$states

  # Return the accuracy of each method
  data.frame(
    filtered_accuracy = accuracy(true_states, filtered_preds),
    smoothed_accuracy = accuracy(true_states, smoothed_preds),
    viterbi_accuracy = accuracy(true_states, viterbi_path)
  )
}

# Store accuracies for each simulation iteration
df_accuracies <- data.frame(
  filtered_accuracy = numeric(),
  smoothed_accuracy = numeric(),
  viterbi_accuracy = numeric()
)
for (i in 1:10) {
  df_accuracies <- rbind(df_accuracies, calculate_accuracy(hmm))
}

# Plot accuracies for each simulation iteration
iterations <- seq_len(nrow(df_accuracies))
plot(
  iterations,
  df_accuracies$filtered_accuracy,
  type = "o",
  col = "blue",
  ylim = c(0, 1),
  xlab = "Simulation iteration",
  ylab = "Accuracy",
  main = "Accuracy across simulation iterations"
)
lines(iterations, df_accuracies$smoothed_accuracy, type = "o", col = "red")
lines(iterations, df_accuracies$viterbi_accuracy, type = "o", col = "green")
legend(
  "bottomright",
  legend = c("Smoothed", "Filtered", "Viterbi"),
  col = c("red", "blue", "green"),
  lty = 1,
  pch = 1
)
```

Question.6

```r
# Calculate entropy for each filtered distribution
filtered_entropy <- apply(
  filtered_probs,
  2,
  entropy.empirical,
  unit = "log2"
)
```

```r
iterations <- seq_along(filtered_entropy)
plot(
  iterations,
  filtered_entropy,
  type = "o",
  xlab = "Observation number",
  ylab = "Entropy",
  main = "Entropy of filtered distributions",
  ylim = c(0, log2(length(states)))
)
```

Question.7

```r
# The filtered distribution at time 100
filtered_prob_t100 <- filtered_probs[, 100]

prob_t101 <- drop(filtered_prob_t100 %*% trans_probs)
print(prob_t101)
```